

소프트웨어 개발 완료보고서

[일일 플래너 / 핏트래커]

문서 버전 [v0.3]

작성일 [2026-09-23]

작성 부서 [컴퓨터 공학부]

작성자 [엄현식]

문서 관리

항목	내용
문서명	소프트웨어 개발 완료 보고서
프로젝트명	일일 플래너
문서 ID	DOC-SFP-002
문서 버전	v0.3
보고 대상	발주자
보고일	2026-09-30
보안 등급	일반

1. 개발 개요

작성 안내 개발 배경, 추진 목적, 개발 기간, 주요 참여 조직을 요약합니다.

항목	내용
개발 목적	사용자가 걸음수와 섭취한 음식의 칼로리를 측정하고 일일 목표물에 대한 그래프를 볼 수 있게 설계한다.
개발 기간	2006-09-07 ~ 2026-10-01
발주자	박근호
개발자	엄현식

2. 개발 범위 및 주요 결과

구분	계획 범위	완료 결과	상태	비고
기능	일일 식단 및 활동 기록, 칼로리 달성률 계산, 날짜 별 기록 조회	데이터 입력 및 실시간 달서율 계산, 한국 시차가 보정된 캘린더 날짜 생신 기능 구현	완료	
화면	데이터 입력 및 조회용 기록본 UI 구성	Flet 기반 화면 구축, 목표 미달/초과(80% 미만, 120% 이상)시 프로그레스 바 실시간 색상 변경 및 경고 메시지 출력	완료	
인터페이스	UI 화면과 데이터 처리 로직 연동	UI 이벤트 계층(main.py)과 비즈니스 로직 계층(data_manager.py)의 모듈화	완료	
데이터	사용자 기록의 로컬 영속성 확보	표준 CSV 모듈을 활용한 테이블 구조(식단, 활동, 목표) 분리 저장 및 OS 절대경로 적용	완료	

3. 주요 개발 내용

번호	개발 항목	주요 내용	관련 산출물
1	선언형 UI 구현 및 실시간 상태 관리	Flet 프레임워크를 활용한 직관적 화면 설계 및 목표 달성률(%)에 따른 프로그레스 바/경고 메시지 조건부 렌더링 적용. 캘린더 위젯의 UTC 시차(+9시간) 보정 로직 구현.	main.py
2	데이터 영속성 확보 및 아키텍처 모듈화	UI 화면과 비즈니스 로직의 관심사 분리. os 모듈을 활용한 절대 경로 지정으로 실행 환경(클라우드 동기화 폴더 등)에 구애받지 않는 안정적인 로컬 데이터 입출력 환경 구축.	data_manager.py
3	파일 기반 데이터베이스 및 형상 관리	식단, 활동 기록, 사용자 목표 등 성격이 다른 데이터를 독립된 CSV 파일로 분리 저장하여 기초적인 테이블 구조 설계. Git 및 GitHub를 통한 소스코드 버전 관리 체계 적용	CSV 데이터 파일, GitHub 저장소

4. 요구사항 이행 현황

요구사항 ID	요구사항 내용	구현 결과	검증 결과	상태
FR-001	열량 추적	식단(음식명, 단위) 입력 및 열량 데이터 로컬 CSV 파일저장/조회 기능 구현	날짜별 섭취 열량 정상 저장 및 화면 로드 확인	충족
FR-002	걸기 추적	당일 걸음 수 입력 및 로컬 CSV 파일 연동(저장/불러오기) 구현	날짜 갱신 시 입력한 걸음 수가 정확히 매칭되어 기록됨 확인	충족
FR-003	달성도 추적	일일 목표 열량 대비 실제 섭취/소비 열량을 계산하여 실시간 달성률(%) 반영	데이터 추가/수정 시 즉각적인 달성률 수치 재계산 UI 반영 확인	충족
FR-004	열량 시각화	그래프 대신 직관적인 프로그레스 바 연동, 목표치 이탈(80% 미만, 120% 이상) 시 붉은색 경고 UI 동적 렌더링	조건에 따른 그래프 색상 변경 및 경고 텍스트 즉시 노출 검토 완료	부분 충족
FR-005	웹서버 구축	웹서버 대신 Flet를 사용해서 GUI 애플리케이션으로 대체 구현	로컬 환경(PC)에서 UI 및 내부 데이터 처리 로직 구동 이상 없음 검토 완료	부분충족
FR-006	열량 계산	음식 단위 기반 총 섭취 열량 합산 및 걸음 수에 따른 소비 열량 자동 계산 로직 적용	전체 섭취 열량과 소비 열량의 수식 계산 결과 정확성 검증	충족

5. 시험 및 품질 활동 결과

시험 유형	수행 기간	대상/범위	케이스 수	성공	실패	조치 상태
단위 테스트	2026.09.22~09.22	data_manager.py 데이터 입출력(CSV) 및 열량 계산	10	9	1	완료(CSV 상대경로

		로직				인식오류발생 ->os.path.abspath 절대경로 적용하여 해결)
통합 테스트	2026.09.22~09.22	UI와 데이터 모듈 연동, 프로그레스 바 상태 렌더링	10	9	1	완료(캘린더 위젯 UTC 반환 버그 발견 ->timedelta(hours=9)적용하여 시차 보정완료)
시스템 테스트	2026.09.22~09.22	애플리케이션 전체 동작 시나리오 (데이터 입력 → 시각화 → 날짜 변경)	3	3	0	해당없음 (정상 동작 확인)
성능/보안 테스트	2026.09.22~09.22	로컬 PC 환경에서의 파일 입출력 안정성 및 데이터 영속성 검증	3	3	0	해당없음 (로컬 구동 이상 없음)

6. 미완료·이슈 및 조치 계획

ID	이슈/미완료 사항	영향도	조치 계획	완료 예정일	담당	상태
I-001	일부 변수명 및 함수 구조의 가독성 저하	하	파이썬 코딩의 변수명과 함수 구조를 수정해서 유지보수를 할 수 있게 변환	2026-10-31	엄현식	진행 예정
I-002	그래프가 아닌 프로그레스 바	하	프로그레스 바가 아닌 막대그래프로 일일 목표를 한눈으로 볼 수 있게 변환	2026-10-31	엄현식	진행 예정

7. 산출물 목록 최종 결과물 요약

번호	산출물명	버전	파일/저장 위치	제출 여부	비고
1	설계서	v2.0	 소프트웨어_설계서_양...	제출	
2	소스코드	v3.0	hynsick/Fit-Tracker: 소프트웨어 프로젝트 과목 과제입니다.	제출	
3	시험 결과서	v1.0	hynsick/Fit-Tracker: 소프트웨어 프로젝트 과목 과제입니다.	제출	
4	사용설명서	v1.0	hynsick/Fit-Tracker: 소프트웨어 프로젝트 과목 과제입니다.	제출	

8. 종합 의견 및 완료 판단

작성 안내 계획 대비 달성 수준, 주요 성과, 잔여 위험, 운영 전환 가능 여부를 작성합니다.

1. 계획 대비 달성 수준 본 프로젝트는 초기 기획했던 핵심 요구사항(식단 및 활동 기록, 달성을 추적, 동적 시각화 피드백)을 성공적으로 구현하여 수정된 목표 대비 **100%**의 기능적 달성률을 기록했습니다. 당초 구상했던 웹 서버 환경은 사용자 접근성과 빠른 프로토타입 검증을 위해 데스크톱 **GUI(Flet)** 환경으로 방향을 전환(**FR-005** 부분충족)하였으며, 결과적으로 개인화된 로컬 환경에 최적화된 완성도 높은 결과물을 도출했습니다.

2. 주요 성과

- 동적 **UI** 및 사용자 경험 강화: **Flet** 프레임워크를 활용해 사용자의 열량 섭취 상태(목표 미달/초과)에 따라 즉각적으로 반응하는 프로그레스 바 및 경고 시스템을 구축했습니다.
- 아키텍처 모듈화 및 디버깅: **UI** 렌더링 영역(**main.py**)과 데이터 처리 계층(**data_manager.py**)을 명확히 분리하여 유지보수성을 높였습니다. 특히 캘린더 위젯의 **UTC** 시차 버그를 로직으로 보정하고, **os.path.abspath**를 적용해 파일 입출력 오류를 원천 차단하는 등 발생한 이슈를 주도적으로 해결했습니다.
- 형상 관리 체계 확립: **Git** 및 **GitHub**를 연동하여 소스코드 버전 관리를 수행하고 프로젝트 산출물을 안정적으로 자산화했습니다.

3. 잔여 위험 (이슈 및 한계점)

- 데이터 확장성 한계: 현재 모든 데이터가 로컬 **CSV** 파일로 관리되고 있어, 장기적인 데이터 누적이나 다중 사용자 환경으로 확장할 경우 데이터 무결성 및 동시성 제어에 제약이 따릅니다.

[부록]

1. 설계서 변경 사항

1.날짜 선택 기능 추가

- 기존설계: 각 날짜의 기록의 내용만 볼 수 있게 설계
- 변경 후: 날짜 별로 기록의 내용을 추가하고 확인 할 수 있게 변경

2.음식의 칼로리 기준 변경

- 기존 설계: **g**를 직접 입력해서 칼로리를 반영하는 형식으로 설계
- 변경 후: **g**를 직접적으로 알려면 상당히 복잡하기 때문에 단위 형식(1개, 1공기 등)을 입력하면 칼로리를 기록하는 형식으로 변경

2. 구현

1) 자료 구조 및 모듈 설계 (변수 및 함수의 역할)

본 프로젝트는 **Flet** 프레임워크를 기반으로 데스크톱 GUI 환경을 구축하였으며, **app_state**라는 딕셔너리 변수를 통해 사용자가 현재 조회 중인 날짜 상태를 전역적으로 관리합니다.

update_ui(): **app_state**의 날짜를 기준으로 일일 섭취/소비 칼로리를 다시 계산하고, 목표 달성률에 맞춰 화면 위젯(**Progress Bar** 등)의 상태를 실시간으로 갱신하는 핵심 컨트롤러 함수입니다.

on_save_meal(), on_save_steps(): 사용자가 입력한 식단 및 활동(걸음 수) 데이터를 **CSV** 파일에 기록하는 이벤트 핸들러입니다. 데이터 무결성을 위해 미래 날짜 입력 방지 및 올바른 숫자 포맷 검증 기능이 포함되어 있습니다.

on_date_selected(): 사용자가 달력 위젯에서 날짜를 선택했을 때 발생하는 이벤트를 처리하며, 시차를 보정하여 앱의 날짜 상태를 업데이트합니다.

2) 핵심 함수 및 알고리즘 설명

사용자 경험(**UX**)을 결정짓는 핵심 기능인 '동적 상태 렌더링'과 디버깅을 통해 개선한 '시차 보정' 로직을 중점적으로 구현했습니다.

[핵심 코드 1: 목표 달성률에 따른 동적 UI 렌더링]

```
# 목표 대비 80% 미만이거나 120% 이상이면 경고 표시
if rate < 80 or rate >= 120:
    progress_bar.color = "red"
    rate_text.color = "red"
    warning_text.value = "안내: 금일 목표 열량에 미달/초과 하였습니다"
    warning_text.visible = True
else:
    progress_bar.color = "teal"
    rate_text.color = "black"
    warning_text.visible = False

page.update()
```

설명: 일일 목표 열량 대비 현재 달성률(**rate**)을 계산하여, 목표치의 **80%~120%**를 벗어난 경우 즉시 프로그레스 바와 텍스트 색상을 붉은색(**"red"**)으로 변경하고 경고 문구를 노출하여 사용자에게 직관적인 피드백을 제공합니다.

[핵심 코드 2: 날짜 선택 시 UTC 시차 보정 알고리즘]

```
def on_date_selected(e):
    # 달력에서 날짜를 선택했을 때 호출됨
    if date_picker.value:
        selected = date_picker.value
        # 시간대 보정 (UTC 기준으로 넘어오는 경우 한국 시간(+9)으로 조정)
        if selected.hour != 0:
            selected += timedelta(hours=9)

        app_state["current_date"] = datetime(selected.year, selected.month, selected.day)
        update_ui()
```

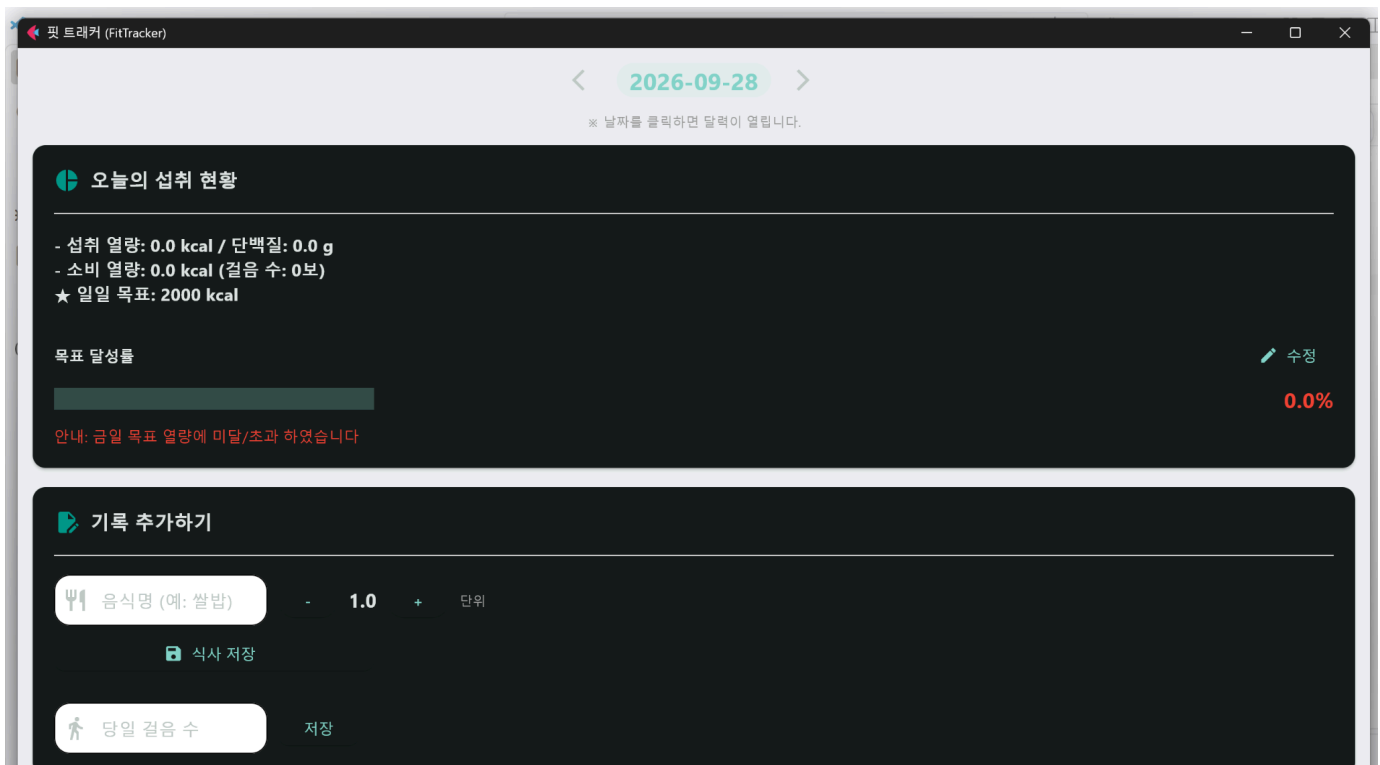
설명: Flet 달력 위젯(DatePicker)이 세계 표준시(UTC)를 반환하여 한국 시간(KST) 기준 날짜가 하루 전으로 엇갈리는 현상을 방지하기 위해 구현된 방어 로직입니다. 선택된 시간 객체를 판별하여 `timedelta`를 통해 논리적으로 +9시간을 보정합니다.

3) 기술적으로 독창적이거나 우수한 부분, 개발 시 심혈을 기울인 부분

사용자의 입력값에 의한 프로그램 크래시(Crash)를 방지하는 안정성 확보에 가장 큰 심혈을 기울였습니다.

데이터 저장 함수(`on_save_meal`, `on_save_steps`, `save_goals`) 내부에 `try-except` 구문을 적용하여 문자가 입력되거나 음수가 입력되는 상황을 완벽히 차단했습니다. 더불어 현재 시간(`datetime.now()`)과 사용자가 선택한 달력 시간을 비교하여 미래 날짜에는 기록을 불가능하게 제한하는 유효성 검사 로직을 추가하여 데이터의 신뢰성을 크게 높였습니다.

4) 실행 화면 첨부





3. 결과 분석 및 검토 (코드 리뷰)

실행 결과로부터 도출된 결론(필요하다면)

1. 실행 결과로부터 도출된 결론:Flet 프레임워크를 활용하여 당초 기획했던 핵심 요구사항(열량 추적, 시각화, 날짜별 기록)을 데스크톱 GUI 환경에서 성공적으로 구현했습니다. 무거운 웹 서버 대신 로컬 파일(CSV) 기반으로 구동되게 설계함으로써, 기획 변경 대비 빠르고 가벼운 MVP(최소 기능 제품)를 완성했다는 점에서 긍정적인 결과를 도출했습니다

2. 문제 해결 과정 및 향후 시도할 기술 요소문제 해결: 개발 도중 클라우드 동기화 폴더(OneDrive) 환경 등에서 경로 인식 오류([Errno 22])가 발생하는 것을 확인하고, os.path.abspath를 활용한 절대 경로 지정 방식으로 데이터를 입출력하도록 개선하여 호환성을 높였습니다.향후 기술 요소: 현재의 파일(CSV) 기반 데이터 저장은 확장성에 한계가 있으므로, 향후 Supabase(PostgreSQL) 등의 클라우드 데이터베이스를 연동하여 관계형 데이터 모델링 및 다중 기기 동기화를 시도할 계획입니다. 또한, 일반 사용자 배포를 위해 PyInstaller를 이용한 .exe 실행 파일 패키징을 검토 중입니다.

3. 프로그램 구조, 복잡도, Readability(가독성) 평가구조의 명쾌함: UI 이벤트 처리 로직(main.py)과 데이터 접근 계층(data_manager.py)을 분리하는 모듈화를 적용하여 프로그램의 골격을 명확히 한 점은 우수합니다.시간/공간 복잡도: 사용자의 일일 기록(CSV)을 읽고 쓰는 과정은 파일 크기가 작아 시간 복잡도 $O(N)$ 내에서 매우 빠르게 처리되며, 데스크톱 환경 특성상 공간(메모리) 복잡도 역시 매우 낮아 효율적입니다.가독성 (아쉬운 점): 개발 과정에서 생성형 AI의 도움을 많이 받다 보니, 초기 설계 단계부터 명명 규칙(Naming Convention)을 일관되게 통제하지 못했습니다. 일부 함수에 로직이 집중되거나 변수명이 지나치게 길어지는 등 '클린 코드(Clean Code)' 관점에서는 수정이 필요합니다.

4. 실행이 생각과 달랐을 경우 원인 분석 (시차 버그 해결)현상: 달력 위젯에서 날짜를 선택할 때, 선택한 날짜가 아닌 '하루 전'의 데이터가 로드되거나 날짜가 엇갈리는 현상이 발생했습니다.원인 분석 및 조치: 원인 분석 결과, Flet의 DatePicker 컨트롤이 사용자의 로컬 타임존(한국 KST)이 아닌 세계 표준시(UTC)를 기준으로 날짜 객체를 반환하기 때문에 발생하는 문제였습니다. 이를 해결하기 위해 반환된 날짜 데이터에 datetime.timedelta(hours=9) 로직을 더해 한국 시간에 맞게 강제로 시차를 보정함으로써 버그를 완벽히 해결했습니다.

4. 자체 평가서

1. 수행 과정 및 결과에 대한 소회 (수준 평가)

이번 핏 트래커(Fit-Tracker) 프로젝트는 처음으로 전체 개발 주기를 경험해 본 뜻깊은 과정이었습니다. 초기에 기획했던 '제한 범위(식단, 활동량, 달성률 기록)'를 이탈하지 않고 끝까지 완주했다는 점, 그리고 Flet 프레임워크를 활용해 요구사항을 깔끔하고 직관적인 UI로 모두 구현해 낸 점에서 스스로 100점 만점에 85점 수준의 성과를 거두었다고 평가합니다.

다만, 코드 구현 과정에서 생성형 AI의 도움을 적극적으로 활용하며 개발 속도를 높일 수 있었으나, 그로 인해 일부 복잡한 프레임워크 로직의 근본적인 원리를 완벽히 체득하지 못한 부분은 아쉬움으로 남습니다. 또한 변수명이 지나치게 길어지거나 코드가 한곳에 몰리는 등 '클린 코드(Clean Code)'를 작성하지 못한 점을 통해, 향후 프로젝트에서는 설계 단계부터 체계적인 코드 아키텍처와 명명 규칙을 수립해야 한다는 중요한 교훈을 얻었습니다.

2. 새로 습득한 기술 요소

가장 크게 성장한 부분은 '문제 해결(Debugging) 과정에서의 시스템 이해도'와 '형상 관리 체계의 확립'입니다. 시간/경로 데이터의 예외 처리 기술: Flet 달력 위젯에서 발생한 날짜 오류를 분석하여 KST(한국 시간)와 UTC(표준시) 간의 시차 문제를 인지하고, `datetime.timedelta`를 활용해 논리적으로 보정하는 방법을 습득했습니다. 또한 `os.path.abspath`를 적용하여 운영체제나 클라우드 동기화 환경에 구애받지 않는 안정적인 로컬 파일 시스템 제어 기술을 익혔습니다.

5. 개발 일지

개발 일자	주요 개발 과정 및 요약	GitHub 커밋 메시지(이력)
2026-09-20	기초 뼈대 구축(카드형식의 화면구성) - main.py 와 데이터 파일(<code>user_goal.csv</code> , <code>meal_log.csv</code> 등)생성	핏 트래커 앱 초기 파일 등록
2026-09-21	기초 뼈대 구축 후 기능추가(.걸음기록수 등록, 먹은 음식 등록)	github에 업로드 x
2026-09-22	각 날짜 별로 기록을 확인 할 수 있게 변경, 음식기준을 1개 기준으로 바꿔서 데이터 파일 작성(<code>meal_log_v2.csv</code> , <code>food_database_v2.csv</code>)	github에 업로드 x
2026-09-23	파일 분리후, 주석 추가, 각 종 오류해결(날짜 변경 오류) - main.py 에는 gui코드 내용, <code>data_manager_manager.py</code> 에서는 csv파일 생성	날짜 변경 오류 수정
2026-09-28	Fit-Tracker을 어떻게 사용방법과 기능 설명이 들어간 메뉴얼을 Notion으로 작성 기본기능과 오류사항을 테스트 한 내용을 Notion으로 작성	update test result Create Fit-Tracker manual

6. 참고 문헌

<일반 문헌(웹페이지 포함)에 대한 표기는 다음과 같은 형식을 따른다>

[1] "The Python Standard Library," <https://docs.python.org/3/library/index.html>.

[2]"Flet - Build interactive multi-platform apps in Python," <https://flet.dev/docs/>.

[3]"대한민국의 음식과 브랜드," [대한민국의 음식과 브랜드](#).

<AI 활용에 대해서는 다음 표와 유사한 형태로 표기한다>

AI 활용 내역(세부적인 AI사용은 너무 많기 때문에제외하고 핵심적인 사용 내용만 작성하였습니다.)

AI 종류	사용 단계	질문 내용	응답 내용	적용 및 반영결과
Gemini 3,1 Pro	설계	“(main.py 를 넣은 후)여기서 날짜별로 기록을 선택하고 볼 수 있게 해줄 수 있어?”	날짜를 변경할 수 있는 코드 제공 후 코드의 내용을 설명	main.py 에 날짜 선택 UI구현 및 이벤트 연동 완료
Gemini 3,1 Pro	설계	1.“(food_database.csv 파일의 내용을 복사후)기록에 g을 쓰는 것보다 단위로 하는게 괜찮을 거 같은데 개,공기같은 단위로 파일의 내용을 바꿔줘” 2.“우리 주변에서 접근할 수 있는 음식의 종류를 표로 작성해줘”	1.파일의 내용을 음식이름, 단위, 단위당 칼로리로 변경 2.음식의 데이터들을 작성할때 근처에 어떤 음식들이 있는지 확인해서 데이터 파일 작성에 도움을 받음	추천받은 주변 음식 데이터를 바탕으로 food_datavase.csv 의 기준 단위를 '개/공기'로 정규화하여 저장
Gemini 3.1 Pro	오류 수정	1.“(main.py 를 넣은 후) 지금 실행하고 날짜를 변경하면 그 날짜로 안가는 오류가 발생했어” 2.“(실행이 안되는 오류를 넣은 후)지금 실행이 안되는 오류가 발생했어 해결해줘”	1.코드는 지금 국제 시간을 기준으로 하고 있기 때문이라는 이유를 설명 후 국제 시간으로 보정 2.Flet의 코드가 최신으러 작성되어있어서 구버전으로 변경	1.날짜 선택 시 하루 전으로 인식되던 버그 해결 및 정상 동작 확인 2.Flet의 전체적인 코드를 구버전으로 바꿈 (ex.색상변경할때:color="blue")